Phytium飞腾

飞腾 RT-Thread 用户使用手册

（V1.0）

飞腾信息技术有限公司
www.phytium.com.cn
2003 年 08 月 03 日

版权所有 © 飞腾信息技术有限公司 2023。 保留一切权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium 和其他飞腾商标均为飞腾信息技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

特别提示

本文档仅作为使用指导，飞腾对本文档内容不做任何明示或暗示的声明或保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

由于产品版本升级或其他原因，本文档内容会不定期进行更新，如有变更，恕不另行通知。

最新技术资源

飞腾嵌入式软件开源社区： https://gitee.com/phytium_embedded

或者访问飞腾软件开发者平台： https://service.phytium.com.cn/developer/

目录

1 RT-Thread 基础 5
1.1 RT-Thread 内核层简介 5
1.2 I/O 设备驱动模型介绍 5
1.2.1 UART 设备 5
1.2.2 PIN 设备 6
1.2.3 SPI/QSPI 设备 7
1.2.4 CAN 设备 8
1.3 RT-Thread 常用内核组件 9
1.4 C++支持 9
1.4.1 支持特性 9
1.4.2 使用方法 10
1.5 SMP 介绍 11
1.6 RT-Smart 简介 11
2 RT-Thread 开发环境 12
2.1 Windows Env 环境配置 12
2.1.1 配置 aarch32 编译环境 12
2.1.2 配置 aarch64 编译环境 12
2.2 Linux 环境配置 13
2.2.1 配置 aarch32 编译环境 13
2.2.2 配置 aarch64 编译环境 14
2.3 Phytium BSP 使用方法 14
2.3.1 重要配置开启方式 14
2.3.2 编译与烧录 17
3 RT-Thread 移植适配 19
3.1 飞腾派 BSP 的制作方法 19
3.2 RT-Thread 内核启动流程（AARCH64） 21
3.3 RT-Thread 驱动适配 26
3.3.1 GPIO 驱动 26

3.3.2 UART 27
3.3.3 SPI 29
3.3.4 QSPI 30
3.3.5 CAN 31
3.3.6 SD 驱动 33
3.3.7 XMAC 驱动 35
3.3.8 中断支持 37
3.4 RT-Thread 内核调试方法 38
4 RT-Thread 综合应用 40
4.1 文件系统与 ulog 日志使用 40
4.1.1 ulog 日志系统 40
4.1.2 文件系统的 POSIX 接口 40
4.1.3 配置 ulog 将日志保存成文件 41
4.2 网络协议与 tftp 服务 42
4.2.1 tftp 协议 42
4.2.2 文件传输演示 42
4.3 RT-Smart 用户态与 Busybox 工具 44
4.3.1 构建 RT-Smart 用户态程序 44
4.3.2 Busybox 工具的使用 44
5 附录 45
A 更新记录 45

1 RT-Thread 基础

RT-Thread，全称是 Real Time-Thread，是一个嵌入式实时多线程操作系统，基本属性之一是支持多任务。在 RT-Thread 系统中，任务通过线程实现的。关于 RT-Thread 的概述和架构介绍。具体内容可参考：

https://www.rt-thread.org/document/site/#/rt-thread-version/rt-thread-standard/README

1.1 RT-Thread 内核层简介

RT-Thread 的内核层是该操作系统的核心部分，提供了实时调度和任务管理等基本功能。RT-Thread 实时内核的实现包括：对象管理、线程管理及调度器、线程间通信管理、时钟管理及内存管理等，具体描述可参考：

https://www.rt-thread.org/document/site/#/rt-thread-version/rt-thread-standard/programming-manual/basic/basic

1.2 I/O 设备驱动模型介绍

RT-Thread 提供了一套简单的 I/O 设备模型框架，它位于硬件和应用程序之间，共分成三层，从上到下分别是 I/O 设备管理层、设备驱动框架层、设备驱动层，具体框架介绍描述可参考：

https://www.rt-thread.org/document/site/#/rt-thread-version/rt-thread-standard/programming-manual/device/device

1.2.1 UART 设备

RT-Thread 提供的 I/O 设备管理接口来访问串口(UART, Universal Asynchronous Receiver/Transmitter）硬件，应用程序可以通过 rt_device_read 等通用接口完成串口硬件的访问。如下所示是访问串口设备的示例，在 BSP 支持 UART2 的基础上，完成串口 UART2 的设置、使能和数据收发。

```
#define SAMPLE_UART_NAME          "uart2"      /* 串口设备名称 */
static rt_device_t serial;                     /* 串口设备句柄 */
struct serial_configure config = RT_SERIAL_CONFIG_DEFAULT;   /* 初始化配置参数 */

/* step1：查找串口设备 */
serial = rt_device_find(SAMPLE_UART_NAME);

/* step2：修改串口配置参数 */
config.baud_rate = BAUD_RATE_9600;            //修改波特率为 9600
```

```
config.data_bits = DATA_BITS_8;                //数据位 8
config.stop_bits = STOP_BITS_1;                //停止位 1
config.bufsz     = 128;                        //修改缓冲区 buff size 为 128
config.parity    = PARITY_NONE;                //无奇偶校验位

/* step3：控制串口设备。通过控制接口传入命令控制字，与控制参数 */
rt_device_control(serial, RT_DEVICE_CTRL_CONFIG, &config);

/* step4：打开串口设备。以中断接收及轮询发送模式打开串口设备 */
rt_device_open(serial, RT_DEVICE_FLAG_INT_RX);

/* 以中断接收及轮询发送模式打开串口设备 */
rt_device_open(serial, RT_DEVICE_FLAG_INT_RX);

/* 发送字符串 */
rt_device_write(serial, 0, str, (sizeof(str) - 1));
```

1.2.2 PIN 设备

RT-Thread 支持应用程序直接控制芯片上的通用引脚（GPIO, General Purpose Input Output），包括通用引脚的电平、输入输出模式等，下所示是访问 PIN 设备的示例，在 BSP 支持引脚 35 的基础上，可以设置引脚模式、设置和获取引脚电平、设置引脚中断响应函数等。

```
#define BEEP_PIN_NUM                35    /* PB0 */

/* 中断回调函数 */
void beep_on(void *args)
{
    rt_kprintf("turn on beep!\n");

    rt_pin_write(BEEP_PIN_NUM, PIN_HIGH);
}

/* 引脚为输出模式 */
rt_pin_mode(BEEP_PIN_NUM, PIN_MODE_OUTPUT);

/* 设置低电平 */
rt_pin_write(BEEP_PIN_NUM, PIN_LOW);
/* 读取引脚电平 */
status = rt_pin_read(BEEP_PIN_NUM);

/* 按键 0 引脚为输入模式 */
rt_pin_mode(KEY0_PIN_NUM, PIN_MODE_INPUT_PULLUP);
```

```
/* 绑定中断，下降沿模式，回调函数名为 beep_on */
rt_pin_attach_irq(KEY0_PIN_NUM, PIN_IRQ_MODE_FALLING, beep_on, RT_NULL);

/* 使能中断 */
rt_pin_irq_enable(KEY0_PIN_NUM, PIN_IRQ_ENABLE);
```

1.2.3 SPI/QSPI 设备

RT-Thread 支持注册 SPI 总线，将 SPI/QSPI 设备挂载在已经注册好的 SPI 总线上，且可以依赖 RT-Thread 的 PIN 框架来绑定 SPI 的片选引脚(cs_pin)，一般 SPI 总线命名原则为 spix，SPI 设备命名原则为 spixy，如 spi10 表示挂载在 spi1 总线上的 0 号设备

```
struct rt_spi_device *spi_device;
spi_device = (struct rt_spi_device *)rt_malloc(sizeof(struct rt_spi_device));
rt_hw_spi_device_attach(spi_device, "spi1", "spi10", GET_PIN(B, 14), RT_NULL);
```

SPI Flash 是常见的 SPI 总线设备，RT-Thread 集成了 SFUD 组件适配不同的 SPI/QSPI Flash，具体的，初始化 SPI Flash，对于定制好的 BSP，这个过程通常会通过 INIT_COMPONENT_EXPORT 导出自动初始化命令完成，应用程序只要获取 SPI Flash 的句柄，填充 rt_spi_message 就可以访问 SPI Flash。使用 QSPI 设备的大体流程也相似。

```
    /* 尝试初始化 SPI Flash，型号为 W25Q128，通常在 BSP 中完成 */
    if (RT_NULL == rt_sfud_flash_probe("W25Q128", "spi10"))
    {
        LOG_E("Failed to probe the W25Q128.");
        return -RT_ERROR;
    };

    #define W25Q_SPI_DEVICE_NAME        "qspi10"    /* SPI 设备名称 */
    struct rt_spi_device *spi_dev_w25q;        /* SPI 设备句柄 */

    /* 查找 spi 设备获取设备句柄 */
    spi_dev_w25q = (struct rt_spi_device
*)rt_device_find(W25Q_SPI_DEVICE_NAME);

    /* 发送命令读取 ID */
    struct rt_spi_message msg1, msg2;

    msg1.send_buf    = &w25x_read_id;
    msg1.recv_buf    = RT_NULL;
    msg1.length      = 1;
    msg1.cs_take     = 1;
```

```
    msg1.cs_release = 0;
    msg1.next       = &msg2;

    msg2.send_buf   = RT_NULL;
    msg2.recv_buf   = id;
    msg2.length     = 5;
    msg2.cs_take    = 0;
    msg2.cs_release = 1;
    msg2.next       = RT_NULL;

    rt_spi_transfer_message(spi_dev_w25q, &msg1);
```

1.2.4 CAN 设备

RT-Thread 支持通过 I/O 设备管理接口来访问 CAN 硬件控制器，相关接口如下所示：

```
rt_device_find          查找设备
rt_device_open          打开设备
rt_device_read          读取数据
rt_device_write         写入数据
rt_device_control       控制设备
rt_device_set_rx_indicate       设置接收回调函数
rt_device_close         关闭设备
```

使用 rt_device_control 接口根据以下命令不同，对 can 设备进行配置

```
#define RT_DEVICE_CTRL_RESUME 0x01 /* 恢复设备 */
#define RT_DEVICE_CTRL_SUSPEND 0x02 /* 挂起设备 */
#define RT_DEVICE_CTRL_CONFIG 0x03 /* 配置设备 */

#define RT_CAN_CMD_SET_FILTER 0x13 /* 设置硬件过滤表 */
#define RT_CAN_CMD_SET_BAUD 0x14 /* 设置波特率 */
#define RT_CAN_CMD_SET_MODE 0x15 /* 设置 CAN 工作模式 */
#define RT_CAN_CMD_SET_PRIV 0x16 /* 设置发送优先级 */
#define RT_CAN_CMD_GET_STATUS 0x17 /* 获取 CAN 设备状态 */
#define RT_CAN_CMD_SET_STATUS_IND 0x18 /* 设置状态回调函数 */
#define RT_CAN_CMD_SET_BUS_HOOK 0x19 /* 设置 CAN 总线钩子函数 */
```

can 消息原型如下

```
struct rt_can_msg
{
 rt_uint32_t id : 29; /* CAN ID，标志格式 11 位，扩展格式 29 位 */
 rt_uint32_t ide : 1; /* 扩展帧标识位 */
rt_uint32_t rtr : 1; /* 远程帧标识位 */
rt_uint32_t rsv : 1; /* 保留位 */
rt_uint32_t len : 8; /* 数据段长度 */
```

```
rt_uint32_t priv : 8; /* 报文发送优先级 */
 rt_uint32_t hdr : 8; /* 硬件过滤表号 */
rt_uint32_t reserved : 8;
rt_uint8_t data[8]; /* 数据段 */
 };
```

用户填充以上结构体，通过 rt_device_write 发送消息

1.3 RT-Thread 常用内核组件

RT-Thread 内置了很多组件以供使用，常用的有网络组件，LIBC，FinSH，虚拟文件系统等等，具体内容介绍可参考：

https://www.rt-thread.org/document/site/#/rt-thread-version/rt-thread-standard/programming-manual/libc/introduction

1.4 C++支持

1.4.1 支持特性

RT-Thread 支持在 C++11 标准下进行开发，支持的主要特性包括，

●Atomic（原子操作）：RT-Thread 支持 C++11 中的原子操作（Atomic），用于在多线程环境下实现无锁同步。开发者可以使用原子操作来实现线程安全的数据访问和操作，避免竞态条件和数据不一致的问题。

●Conditional variables（条件变量）：RT-Thread 支持 C++11 中的条件变量，用于实现线程之间的同步与通信。条件变量可以让线程在特定条件下等待或被唤醒，实现线程间的协调与同步。

●Clocks（时钟）：RT-Thread 支持 C++11 中的时钟函数和类型，如 std::chrono 命名空间中的高精度时钟类型和时钟函数。这使得开发者可以测量和控制代码的执行时间，实现精确的时间计算和时间相关的操作。

●Future：RT-Thread 支持 C++11 中的 Future，用于实现异步编程模型。开发者可以通过 Future 和 Promise 机制，从异步操作中获得返回值或异常，并进行更加灵活的线程间通信与任务调度。

●Mutexes（互斥锁）：RT-Thread 支持 C++11 中的互斥锁（Mutex），用于实现对临界区的互斥访问。通过互斥锁，可以保证对共享资源的线程安全访问，避免竞争条件和数据损坏。

●Threads（线程）：RT-Thread 支持 C++11 中的线程库，开发者可以使用 std::thread 创建和管理线程。通过线程库，可以方便地创建新线程、设置线程属性、进行线程同步和线程间通信等操作。

●TLS（线程本地存储）：RT-Thread 支持 C++11 中的线程本地存储（TLS），用于在多线程环境下实现线程私有的全局变量。每个线程拥有自己独立的 TLS 变量，避免了全局变量在多线程环境下的竞态

条件问题。

1.4.2 使用方法

注：在 RT-Thread 中使用 C++11，需要修改工具链中的部分文件。请在修改之前，备份好工具链。

1) 使能 C++11

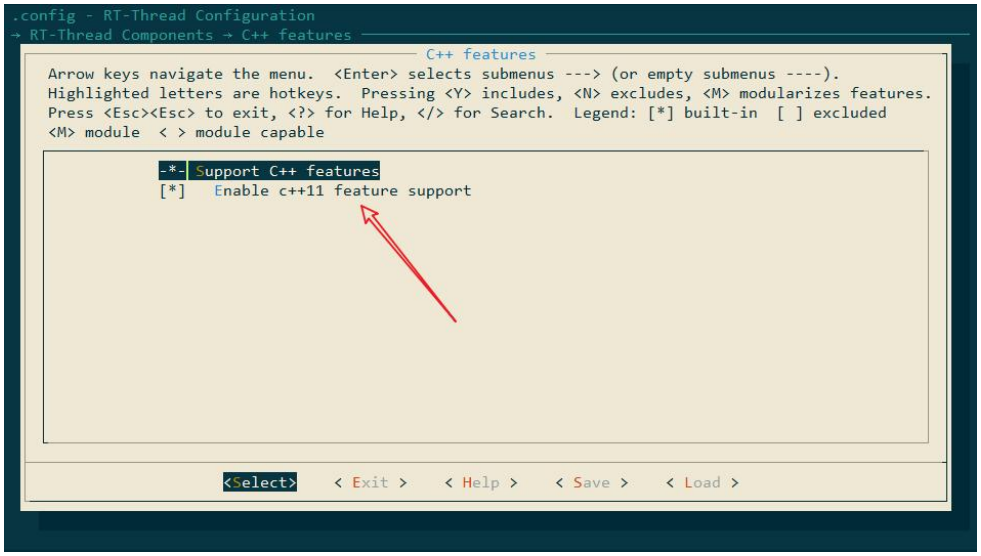


图 1.2 C++使能

2) 下载 GCC 工具链

```
gcc version 10.2.1 20201103 (release) (GNU Arm Embedded Toolchain 10-2020-q4-major)
```

3) 删除下面的文件

```
rm -f toolchain/arm-none-eabi/include/c++/10.2.1/thread
rm -f toolchain/arm-none-eabi/include/c++/10.2.1/mutex
rm -f toolchain/arm-none-eabi/include/c++/10.2.1/condition_variable
rm -f toolchain/arm-none-eabi/include/c++/10.2.1/future
rm -f toolchain/arm-none-eabi/include/pthread.h
```

4) 清除下面文件内容，保留文件避免编译失败

```
toolchain/arm-none-eabi/include/sys/_pthreadtypes.h
```

5) 更新 rtconfig.py,添加 C++编译参数

```
CXXFLAGS = CFLAGS + ' -std=c++11 -fabi-version=0 -MMD -MP -MF'
```

6) 注意事项

RT-Thread RTOS 中的 C++组件实现了基本的环境，如 new/delete 运算符。

由于 RT-Thread RTOS 主要用于嵌入式系统，对于 C++应用程序有一些规则：

- 不使用异常。
- 不使用运行时类型信息(RTTI)。
- 模板是不鼓励的，因为它容易导致代码冗长。
- 不鼓励使用静态类变量。调用它们的构造函数的时间和位置无法精确控制，这会使多线程编程变得非常复杂。
- 不鼓励多重继承，因为它会导致难以处理的混乱情况。

1.5 SMP 介绍

SMP:对称多处理（Symmetrical Multi-Processing）简称 SMP，是指在一个计算机上汇集了一组处理器(多 CPU),各 CPU 之间共享内存子系统以及总线结构。

RT-Thread 自 v4.0.0 版本开始支持 SMP，在对称多核上可以通过使能 RT_USING_SMP 来开启，具体内容可参考：

https://www.rt-thread.org/document/site/#/rt-thread-version/rt-thread-standard/programming-manual/smp/smp

1.6 RT-Smart 简介

RT-Thread Smart 是基于 RT-Thread 操作系统上的混合操作系统，简称为 rt-smart，它把应用从内核中独立出来，形成独立的用户态应用程序，并具备独立的地址空间（32 位系统上是 4G 的独立地址空间）。RT-Smart 整体结构采用了内核态和用户态的划分，通过 MMU 和系统调用的方式将系统区分为内核态和用户态。具体内容可参考：

https://www.rt-thread.org/document/site/#/rt-thread-version/rt-thread-smart/introduction/rt-smart-intro/rt-smart-intro

2 RT-Thread 开发环境

RT-Thread 支持在 Windows Env 和 Ubuntu x86_64 环境下进行开发，本章节对编译环境搭建和开发方式进行详细描述。

2.1 Windows Env 环境配置

首先需要在 RT-Thread 官网安装 RT-Thread Env 工具，安装链接位于官网上方“资源->下载”中，安装完成后分别进行 aarch32 与 aarch64 环境配置。

2.1.1 配置 aarch32 编译环境

1) 在 aarch32 目录下创建 tools 目录，后续用于存放 RT-Thread 编译工具链

```
cd      ./aarch32
mkdir   tools
```

2) 在 tools 目录下下载两个 python 脚本，get_toolchain.py 和 ci.py，下载完后给两个脚本添加执行权限

```
cd      ./tools
wget    https://gitee.com/rtthread/ART-Pi-smart/raw/master/tools/get_toolchain.py
wget    https://gitee.com/rtthread/ART-Pi-smart/raw/master/tools/ci.py
chmod +x get_toolchain.py ci.py
```

3) 然后运行 get_toolchain.py 脚本，拉取 aarch32 交叉编译链 arm-linux-musleabi_for_x86_64-pc-linux-gnu

```
python3 ./get_toolchain.py arm
```

4) 在 aarch32 目录下下载脚本 smart-env.bat，然后运行脚本生效环境变量

```
cd      ./aarch32
wget    https://gitee.com/rtthread/ART-Pi-smart/raw/master/smart-env.bat
./smart-env.bat arm
```

5) 如下所示是 aarch32 编译相关的环境变量，运行 scons 前要确保环境变量设置正确

```
source ./smart_env.sh
Arch     => arm
CC       => gcc
PREFIX   => arm-linux-musleabi-
EXEC_PATH => /mnt/d/phytium/project/rt-thread/rt-smart/rt-thread-dev/bsp/phytium/aarch32/tools/gnu_gcc/arm-linux-musleabi_for_x86_64-pc-linux-gnu/bin
```

图 2.1 aarch32 环境变量

2.1.2 配置 aarch64 编译环境

1) 在 aarch64 目录下创建 tools 目录，后续用于存放 RT-Thread 编译工具链

```
cd      ./aarch64
mkdir   tools
```

2) 在 tools 目录下下载两个 python 脚本，get_toolchain.py 和 ci.py，下载完后给两个脚本添加执行权限

```
cd      ./tools
wget    https://gitee.com/rtthread/ART-Pi-smart/raw/master/tools/get_toolchain.py
wget    https://gitee.com/rtthread/ART-Pi-smart/raw/master/tools/ci.py
chmod +x get_toolchain.py ci.py
```

3) 然后运行 get_toolchain.py 脚本，拉取 aarch64 交叉编译链`aarch64-linux-musleabi_for_x86_64-pc-linux-gnu`

```
python3 ./get_toolchain.py aarch64
```

4) 在 aarch64 目录下下载脚本 smart-env.bat

```
cd      ./aarch64
wget    https://gitee.com/rtthread/ART-Pi-smart/raw/master/smart-env.bat
```

5) 修改 smart-env.bat 文件内容 如下图所示

```
@set RTT_CC=gcc
@set RTT_EXEC_PATH=%cd%\tools\gnu_gcc\aarch64-linux-musleabi_for_i686-w64-mingw32\bin
@set RTT_CC_PREFIX=aarch64-linux-musleabi-
@set PATH=%RTT_EXEC_PATH%;%ENV_ROOT%\tools\gnu_gcc\arm_gcc\mingw\bin;%PATH%
```

图 2.2 aarch64 smart-env.bat 修改

6) 运行脚本生效环境变量

2.2 Linux 环境配置

2.2.1 配置 aarch32 编译环境

1) 在 aarch32 目录下创建 tools 目录，后续用于存放 RT-Thread 编译工具链

```
cd      ./aarch32
mkdir   tools
```

2) 在 tools 目录下下载两个 python 脚本，get_toolchain.py 和 ci.py，下载完后给两个脚本添加执行权限

```
cd      ./tools
wget    https://gitee.com/rtthread/ART-Pi-smart/raw/master/tools/get_toolchain.py
wget    https://gitee.com/rtthread/ART-Pi-smart/raw/master/tools/ci.py
chmod +x get_toolchain.py ci.py
```

3) 然后运行 get_toolchain.py 脚本，拉取 aarch32 交叉编译链 arm-linux-musleabi_for_x86_6

4-pc-linux-gnu

```
python3 ./get_toolchain.py arm
```

4) 在 aarch32 目录下下载脚本 smart-env.sh，然后运行脚本生效环境变量

```
cd      ./aarch32
wget    https://gitee.com/rtthread/ART-Pi-smart/raw/master/smart-env.bat
./smart-env.bat sh
```

5) 如下所示是 aarch32 编译相关的环境变量，运行 scons 前要确保环境变量设置正确

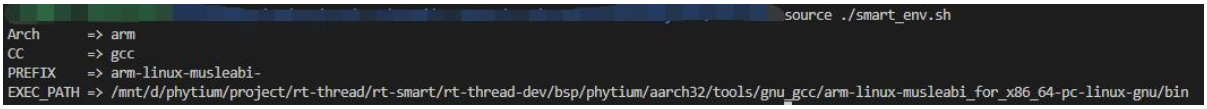


图 2.3 aarch32 环境变量

2.2.2 配置 aarch64 编译环境

1) 在 aarch64 目录下创建 tools 目录，后续用于存放 RT-Thread 编译工具链

```
cd      ./aarch64
mkdir   tools
```

2) 在 tools 目录下下载两个 python 脚本，get_toolchain.py 和 ci.py，下载完后给两个脚本添加执行权限

```
cd      ./tools
wget    https://gitee.com/rtthread/ART-Pi-smart/raw/master/tools/get_toolchain.py
wget    https://gitee.com/rtthread/ART-Pi-smart/raw/master/tools/ci.py
chmod +x get_toolchain.py ci.py
```

3) 然后运行 get_toolchain.py 脚本，拉取 aarch64 交叉编译链`aarch64-linux-musleabi_for_x86_64-pc-linux-gnu`

```
python3 ./get_toolchain.py aarch64
```

4) 在 aarch64 目录下下载脚本 smart-env.sh，然后运行脚本生效环境变量

```
cd      ./aarch64
wget    https://gitee.com/rtthread/ART-Pi-smart/raw/master/smart-env.bat
./smart-env.bat sh
```

2.3 Phytium BSP 使用方法

2.3.1 重要配置开启方式

1) 如何选择芯片

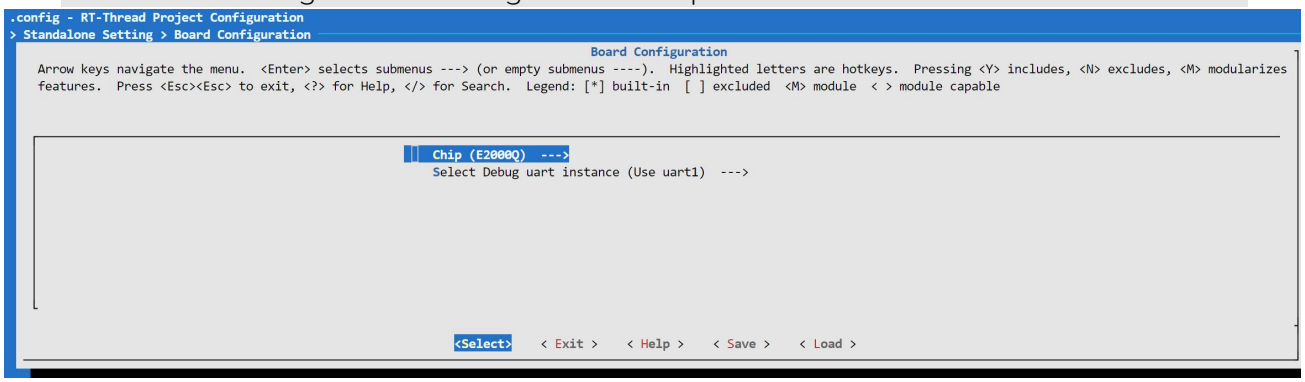
以 E2000Q RT-Smart 为例，Linux 环境下，运行 make load_e2000q_rtsmart 加载默认的 rtconfig,

然后输入下列命令，进入 menuconfig 进一步配置，

Linux：scons --menuconfig
Windows Env：menuconfig

开发者通过以下选择进行配置

Standalone Setting > Board Configuration > Chip



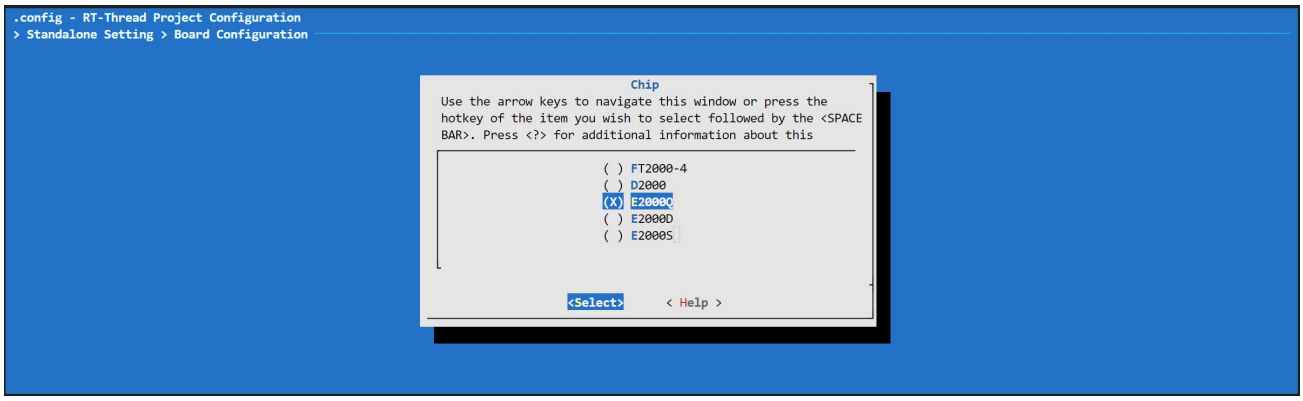


图 2.4 芯片选择

2) 如何选择驱动

Hardware Drivers > On-chip Peripheral Drivers

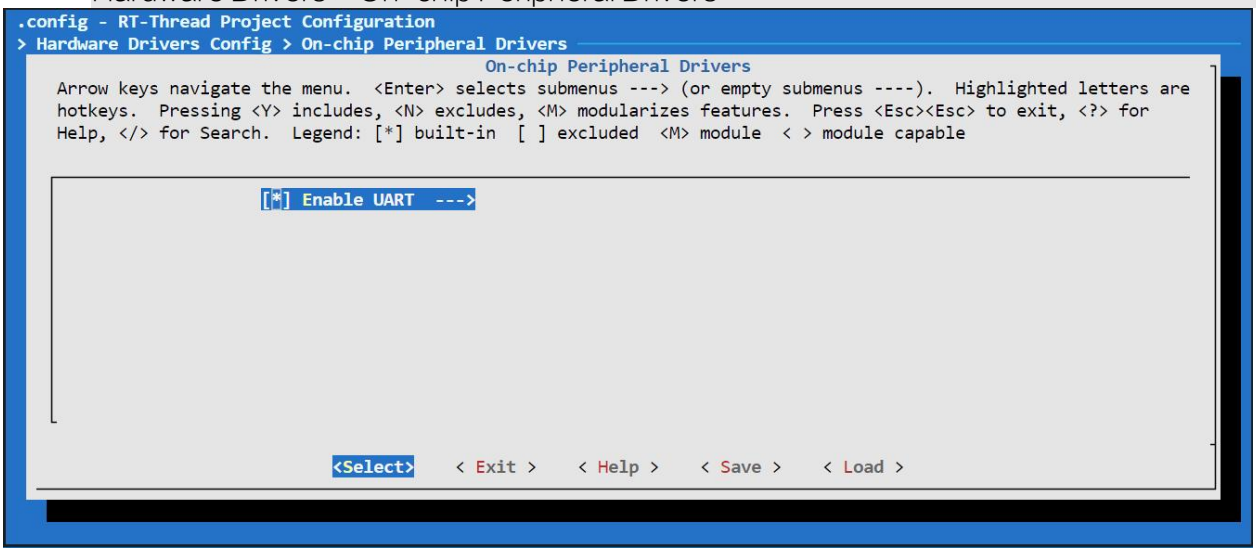


图 2.5 驱动选择

3) 如何开启 SDK 内部调试信息

开发者通过以下选项进行调试信息等级的设置

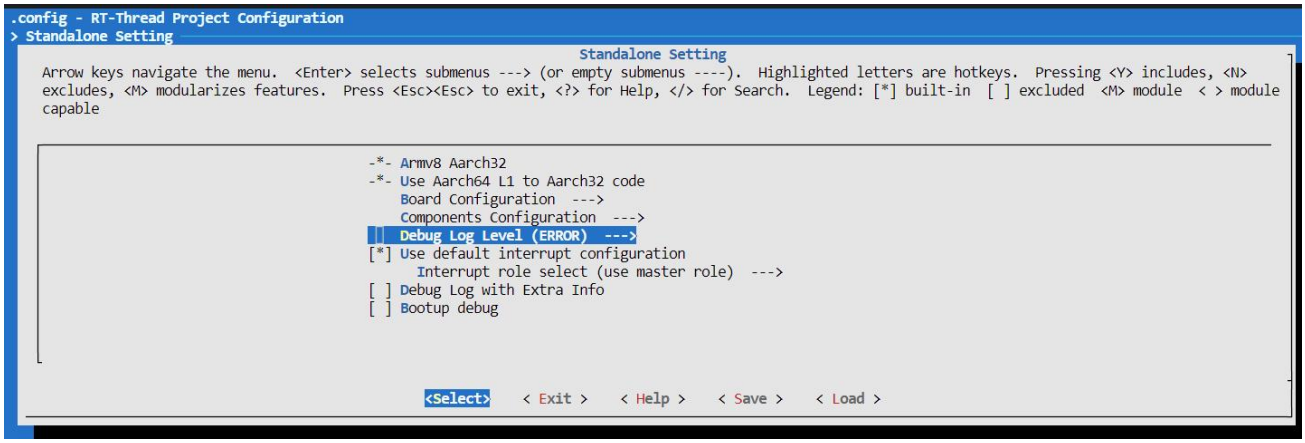


图 2.6 调试等级选择

4) 如何切换 RT-Smart 工作模式

开发者通过以下配置开启 RT-Thread Smart 功能

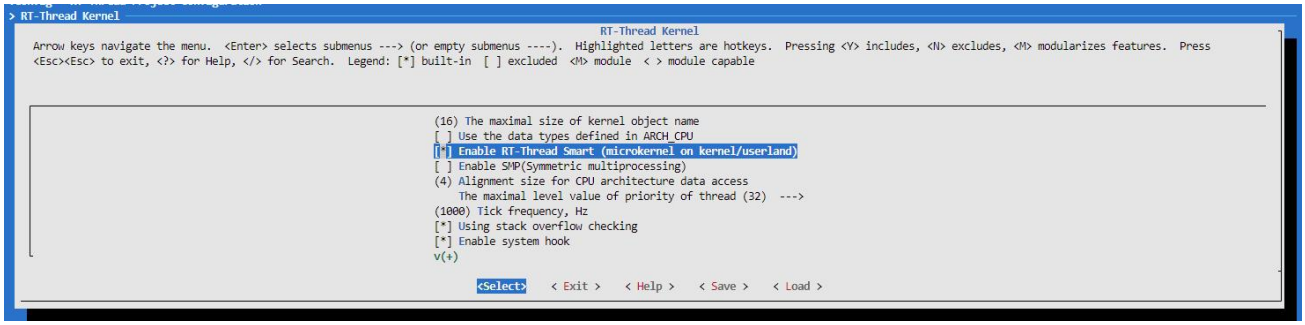


图 2.7 开启 RT-Smart

5) 如何开启 SMP

开发者通过以下配置开启 SMP

```
                                  RT-Thread Kernel
Arrow keys navigate the menu.  <Enter> selects submenus ---> (or empty submenus ----).  Highlighted letters are hotkeys.
Pressing <Y> includes, <N> excludes, <M> modularizes features.  Press <Esc><Esc> to exit, <?> for Help, </> for Search.
Legend: [*] built-in  [ ] excluded  <M> module  < > module capable

                    (16) The maximal size of kernel object name
                    [ ] Use the data types defined in ARCH_CPU
                    [ ] Enable RT-Thread Smart (microkernel on kernel/userland)
                    [*] Enable SMP(Symmetric multiprocessing)
                    (2)   Number of CPUs (NEW)
                    (4) Alignment size for CPU architecture data access
                        The maximal level value of priority of thread (32)  --->
                    (100) Tick frequency, Hz
                    [*] Using stack overflow checking
                    [*] Enable system hook
                    [*]   Using function pointers as system hook
                    -*- Enable IDLE Task hook
                    (4)   The max size of idle hook list
                    (40960) The stack size of idle thread
                    (40960) The stack size of system thread (for defunct etc.) (NEW)
                    -*- Enable software timer with a timer thread
                    (4)   The priority level value of timer thread
                    (4096) The stack size of timer thread
                        kservice optimization  --->
                    [*] Enable debugging features  --->
                        Inter-Thread communication  --->
                    ↓(+)

                        <Select>    < Exit >    < Help >    < Save >    < Load >
```

图 2.8 SMP 配置选项

2.3.2 编译与烧录

1) 执行以下指令进行编译

```
scons -c    /*清除上次编译结果*/
scons       /*进行编译*/
```

2) 完成编译之后目录下将生成以下几个文件

```
rtthread_a32.bin
rtthread_a32.elf
rtthread_a32.map
```

3) 启动镜像程序

a) 可以用串口通过 XMODEM 协议将 bin/elf 文件上传到开发板，连接开发板后,输入以下命令

```
Loadx 80080000
```

```
E2000#loadx 0x80080000
## Ready for binary (xmodem) download to 0x80080000 at 115200 bps...
C
Starting xmodem transfer.  Press Ctrl+C to cancel.
Transferring rtthread_a64.bin...
    3%      10 KB      10 KB/sec    00:00:32 ETA    0 Errors
```

图 2.9 loadx_cmd

b) 上传完成后输入以下命令启动

```
go 80080000
```

```
 \ | /
- RT -     Thread Operating System
 / | \     5.0.1 build Jun 19 2023 16:02:06
 2006 - 2022 Copyright by RT-Thread team
msh />
msh />
msh />
msh />
```

图 2.10 go_cmd

3 RT-Thread 移植适配

3.1 飞腾派 BSP 的制作方法

飞腾派的 RT-Thread BSP 支持 AARCH64 和 AARCH32 两种模式，同时支持 SMP 和 RT-Smart。本章节介绍了飞腾派 BSP 的制作过程，旨在帮助开发者理解飞腾派 BSP 的框架结构，更方便快捷地制作新的飞腾派 BSP。

飞腾派 RT-Thread BSP 的主要功能由 Phytium Standalone SDK（后续称 Standalone SDK）提供，相关源码可以通过 Gitee 链接获取(https://gitee.com/phytium_embedded/phytium-standalone-sdk)。Standalone SDK 是为飞腾系统 CPU 的嵌入式 RTOS 系统设计提供的简单易用，可裁剪可以拓展的裸机驱动，可以用于导出给各类 RTOS 制作 BSP。

如下图所示是飞腾派 RT-Thread BSP 目录，包含一系列目录，其中 aarch64 和 aarch32 目录分别提供执行模式相关的基础支持，board 目录提供飞腾派的基础支持，libaries 包含 Phytium Standalone SDK 的源码和飞腾派的 RT-Thread 外设驱动，tools 提供的脚本用于支持 BSP 的使用。

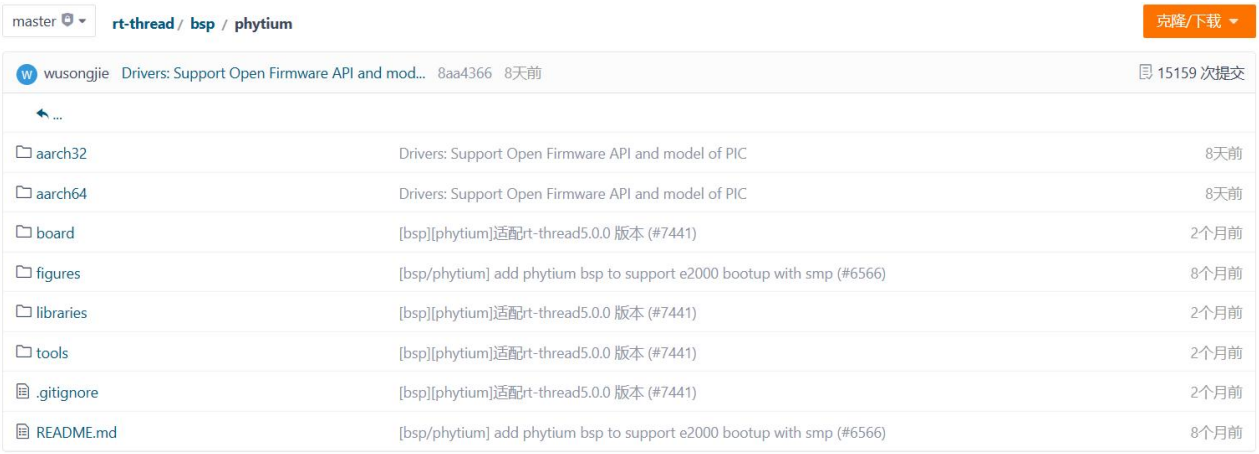


图 3.2 飞腾派 RT-Thread BSP

表 3.1 飞腾派 RT-Thread BSP 主要内容

项目	文件夹	说明
通用库	/libraries/standalone	phytium standalone sdk 的源码，包括了飞腾派的硬件参数标定，硬件驱动

		等
	/libraries/drivers	适配 phytium standalone sdk 的 rt-thread 驱动，如 drv_usart.c 是串口驱动，drv_sdio.c 是 SD 控制器驱动
	/libraries/examples	飞腾派 rt-thread bsp 自带的例程
AARCH64 支持	/aarch64/applications	64 位执行模式下 rt-thread 内核的 C 入口函数和文件系统挂载流程，包括 main.c 和 mnt.c
	/aarch64/configs	64 位执行模式下运行 rt-thread 和 rt-smart 的配置，可以加载成 rtconfig.h 作为编译配置，包括 firefly_rthread_smp、firefly_rtsmart_smp
AARCH32 支持	/aarch32/applications	64 位执行模式下 rt-thread 内核的 C 函数入口
	/aarch32/boot	从 AARCH64 切换到 AARCH32 模式启动的相关实现
	/aarch32/configs	32 位执行模式下运行 rt-thread 和 rt-smart 的配置
飞腾派配置	/board/firefly	飞腾派的相关参数，内存布局设置等
	/board/board.c	飞腾派给 rt-thread 内核的配置参数，内存设置
	/board/phytium_cpu.c	飞腾派的 CPU 中断等配置参数
	/board/secondary_cpu.c	飞腾派的 SMP 支持

上面是 RT-Thread 基线中的飞腾派 BSP 情况介绍，如果准备制作一个新的飞腾派 BSP，需要先储备以下的知识，本章节的后面小节都会围绕这些知识展开

1) RT-Thread 构建工具 scons 和 Kconfig 的使用
   a) RT-Thread 使用 scons 作为系统的构建工具，因此了解 Scons 的常用命令对制作新 BSP 是基本要求，详细的内容可以参考 RT-Thread 文档中心关于 scons 的介绍(https://www.rt-thread.org/document/site/#/development-tools/scons/scons);
   b) RT-Thread 系统通过 menuconfig 的方式进行配置,而 menuconfig 中的选项是由 Kconfig 文件决定的，想要对配置 RT-Thread 系统，需要对 Kconfig 语法有一定了解,具体内容见

(https://www.rt-thread.org/document/site/#/development-tools/build-config-system/Kconfig)

2) RT-Thread 的内核启动流程
   a) 制作 RT-Thread 的新系统需要熟悉内核的启动流程，RT-Thread 的初始化按照不同硬件平台类型实现在 libcpu 目录中，飞腾派的 AARCH64 模式通过/libcpu/aarch64 实现支持，AARCH32 模式通过/libcpu/arm/cortex-a 实现支持；
   b) AARCH64 和 AARCH32 的启动流程有相当部分是用汇编实现的，需要对 ARM 的汇编语言由一定的了解，能够看懂寄存器操作和程序跳转流程，详细内容可以参考 ARM 汇编手册(https://developer.arm.com/documentation/dui0473/c/writing-arm-assembly-language/unified-assembler-language?lang=en)；
   c) 启动流程中涉及到对许多体系结构相关的设置，如异常模型、内存模型的配置，cache、mmu 的初始化，RT-Smart 初始化中还设计虚拟地址页表的建立，制作新 BSP 遇到问题时需要对相关概念有一定的理解；
3) RT-Thread 设备驱动开发方法
   a) 在 RT-Thread 系统中，应用程序通过设备驱动框架来操作硬件，因此了解设备驱动框架，对添加 BSP 驱动是很重要的；
   b) 设备驱动开发前，需要熟悉硬件的特性，以及硬件控制寄存器的功能，相关内容可以参考飞腾派软件编程手册；
4) Phytium Standalone SDK 的使用与移植
   a) 飞腾派的硬件设备驱动底层需要依赖 Standalone SDK，制作新 BSP 需要熟悉 Stanalone SDK 的使用，理解相关 API 的功能；
   b) RT-Thread 的硬件设备驱动需要注册给设备驱动框架使用，在这一步需要用 Standalone SDK 的 API 实现驱动；
5) RT-Thread 内核调试方法
   a) 初步适配完成后，系统可能已经跑起来了，但是总会遇到各种大大小小的问题，最后介绍了常用的 RT-Thread 内核调试方法，也列举了一些常见的问题及其解决方法。

3.2 RT-Thread 内核启动流程（AARCH64）

了解 RT-Thread 内核的启动流程对于移植适配 BSP 以及定位问题都是很有意义的，本节以 AARCH64 模式下的 RT-Thread 内核启动为例进行介绍。

1) 从 u-boot 跳转到 rt-thread
   a) 飞腾派的 RT-Thread 从 u-boot 环境下启动，对内存、外设等进行必要的初始化工作由 u-boot 及之前的固件完成；
   b) 通过命令 go 0x80080000 从 u-boot 跳转到 rt-thread，执行 rt-thread 的第一行代码
2) rt-thread 的第一行代码由链接脚本指定
   a) 启动点由飞腾派 BSP 的链接脚本 aarch64\links.lds(RT-Thread) 或者 aarch64\links_smart.lds(RT-Smart)；
   b) aarch64 模式下，启动后运行的第一行代码在 libcpu\aarch64\cortex-a\entry_point.S 中；
   c) 启动后，为了避免之前固件中的读写动作没有同步，可以选择手动进行 flush cache 操作。

```
/* link.lds */
SECTIONS
{
    . = 0x80080000 ;
    . = ALIGN(4096);
    .text :
    {
        KEEP(*(.text.entrypoint))        /* The entry point */
        *(.vectors)
        *(.text)                         /* remaining code */
        *(.text.*)                       /* remaining code */
```

```
/* entry_point.S */
section ".text.entrypoint","ax"
.global __start

__start:
#ifdef ARCH_ARM_BOOTWITH_FLUSH_CACHE
    bl      __asm_flush_dcache_all
#endif
```

3) 单核启动后切换 EL 等级，清理执行环境
   a) 启动后首先要确定当前核心，主核启动，首先将其它核设置为 idle 睡眠状态，后面如果 SMP 使能了，主核启动完成后会唤醒其他核；
   b) 主核开始启动，先判断当前的 EL,如果启动后是 EL3 或者 EL2，先将核心切换到 EL1;
   c) 然后将 bss 段清零，设置 sp 指针，准备一个干净的执行环境。
4) 单初始化 mmu 和 cache

a) rt-thread 和 rt-smart 模式下设置不同的初期启动页表项，填入 ttbr 寄存器；

b) 使能 mmu，同时打开 icache 和 dcache

5) 跳转到 c 函数，继续进行初始化

a) 跳转到 c 函数 rtthread_startup，这个函数实现在/src/components.c 中，后续的 rt_hw_board_init 由 BSP 实现，会进一步完成初始化

```
/**
 * @brief   This function will call all levels of initialization functions to complete
 *          the initialization of the system, and finally start the scheduler.
 *
 * @return Normally never returns. If 0 is returned, the scheduler failed.
 */
int rtthread_startup(void)
{
    rt_hw_interrupt_disable();

    /* board level initialization
     * NOTE: please initialize heap inside board initialization.
     */
    rt_hw_board_init();

    /* show RT-Thread version */
    rt_show_version();
```

6) BSP 完成板级初始化

a) rt_hw_board_init 由 BSP 实现，会进行和板子相关的初始化，如内存布局，页表，堆的初始化

b) 在上述初始化完成后，初始化中断，然后调用 rt_components_board_init，这个函数会调用运行 INIT_BOARD_EXPORT 注册的函数，完成各个驱动的初始化和注册

c) 串口是启动后开发人员与板子交互的主要接口，这里将控制台和串口绑定，后面就可以通过串口完成输入输出了

```
    /* shell init */
    #if defined(RT_USING_CONSOLE) && defined(RT_USING_DEVICE)
        /* set console device */
        rt_console_set_device(RT_CONSOLE_DEVICE_NAME);
    #endif
```

7) 驱动程序的注册与初始化

a) 在 BSP 初始化的先后，串口、SD 卡控制器等硬件设备的初始化也会前后开始，在 rt_components_init 中完成；

b) 驱动设备要在内核启动过程中自动初始化，需要将驱动初始化函数注册给 RT-Thread，RT-Thread 为此提供了一系列宏，规定了几个初始化阶段，其中用 INIT_BOARD_EXPORT 注册的驱动最先初始化，如串口驱动，通用定时器等，系统的最基础功能依赖这些驱动；其他驱动一般用 INIT_DEVICE_EXPORT 注册就可以了，如 SD,SPI 控制器；然后才是 SD 卡、SPI Flash 乃至文件系统这些的初始化，用 INIT_COMPONENT_EXPORT 完成，它们必须等到基础硬件初始化完成；

```
/* board init routines will be called in board_init() function */
#define INIT_BOARD_EXPORT(fn)                INIT_EXPORT(fn, "1")

/* pre/device/component/env/app init routines will be called in init_thread */
/* components pre-initialization (pure software initialization) */
#define INIT_PREV_EXPORT(fn)                 INIT_EXPORT(fn, "2")
/* device initialization */
#define INIT_DEVICE_EXPORT(fn)               INIT_EXPORT(fn, "3")
/* components initialization (dfs, lwip, ...) */
#define INIT_COMPONENT_EXPORT(fn)            INIT_EXPORT(fn, "4")
/* environment initialization (mount disk, ...) */
#define INIT_ENV_EXPORT(fn)                  INIT_EXPORT(fn, "5")
/* application initialization (rtgui application etc ...) */
#define INIT_APP_EXPORT(fn)                  INIT_EXPORT(fn, "6")
```

8) SMP 多核初始化

a) rt_hw_board_init 中最后一般是启动多核，首先为每个核心注册一个 idle 回调函数，让核心在 idle 时处于睡眠状态，然后设置核间 IPI 中断，这些都是后面 SMP 核间通信中需要用到的；

b) 前面一系列初始化操作完成后，会进入 rt_application_init 启动 RT-Thread 的主线程，主线程会调用 BSP 实现的函数 rt_hw_secondary_cpu_up 启动主核之外的的其他核心；

c) BSP 中的 rt_hw_secondary_cpu_up 负责依次启动从核，主要的操作是通过 psci 传递各个从核需要执行的函数入口，然后启动各个从核，其中_secondary_cpu_entry 是 entry_point.S 中由 RT-Thread 实现的。

```
void rt_hw_secondary_cpu_up(void)
{
    rt_uint32_t i;
    rt_uint32_t cpu_mask = 0;
    int cpu_id;
    cpu_id = rt_hw_cpu_id();
```

```
        rt_kprintf("rt_hw_secondary_cpu_up is processing \r\n");
        for (i = 0; i < RT_CPUS_NR;i++)
        {
            if(i == cpu_id)
            {
                continue;
            }
            cpu_mask = 1 << phytium_cpu_id_mapping(i);


    #if defined(TARGET_ARMV8_AARCH64)
            /* code */
            char *entry = (char *)_secondary_cpu_entry; /*  从核启动函数  */
            entry += PV_OFFSET;
            PsciCpuOn(cpu_mask, (uintptr)entry);
            __DSB();
    #else
                /* code */
            PsciCpuOn(cpu_mask, (uintptr)rt_secondary_cpu_entry);
            __asm__ volatile("dsb" ::: "memory");
    #endif

        }
    }
```

9) 内核启动完成，进入 main 函数

a) 前面讲到 RT-Thread 的主线程启动，主线程在完成 SMP 多核启动后会调用 main 函数，main 函数之后开发者就可以按照需要开发自己的功能了

```
void main_thread_entry(void *parameter)
{
    extern int main(void);

#ifdef RT_USING_COMPONENTS_INIT
    /* RT-Thread components initialization */
    rt_components_init();
#endif /* RT_USING_COMPONENTS_INIT */

#ifdef RT_USING_SMP
    rt_hw_secondary_cpu_up();
#endif /* RT_USING_SMP */
    /* invoke system main function */
```

```
        main();
    }
```

b) 飞腾派 BSP 中在 main.c 中演示了如何为每个核心创建一个任务，然后绑定运行在指定核心上

3.3 RT-Thread 驱动适配

3.3.1 GPIO 驱动

飞腾派的 GPIO 可以使用 RT-Thread 的 PIN 设备进行驱动，描述一个 PIN 设备需要控制器编号、port 编号与引脚号，E2000 上 port 默认为 A 组即 0 号 port，下面简要描述适配 PIN 设备的过程：

1) 使用宏定义给飞腾派的 GPIO 引脚进行编号定义，后面 PIN 设备的索引就使用这个编号

```
/* 使用 uint32_t 类型编码引脚，bit[12:19] 定义引脚控制器编号，bit[8:11] 定义引脚的 port 号，bit[0:7] 定义引脚引脚号 */
#define FGPIO_OPS_PIN_INDEX(ctrl, port, pin)        SET_REG32_BITS(ctrl, 19, 12) | \
                                                    SET_REG32_BITS(port, 11, 8) | \
                                                    SET_REG32_BITS(pin, 7, 0)
#define FGPIO_OPS_PIN_CTRL_ID(pin_idx)              GET_REG32_BITS(pin_idx, 19, 12)
#define FGPIO_OPS_PIN_PORT_ID(pin_idx)              GET_REG32_BITS(pin_idx, 11, 8)
#define FGPIO_OPS_PIN_ID(pin_idx)                   GET_REG32_BITS(pin_idx, 7, 0)
```

1) 创建数据结构，封装 Standalone 驱动的 GPIO 控制器和 GPIO 引脚实例，作为 PIN 设备的私有数据使用

```
typedef void (*FGpioOpsIrqHandler)(s32 vector, void *param);
typedef struct
{
    FGpioDirection direction; /* 引脚方向 */
    boolean en_irq; /* 是否使能中断 */
    FGpioIrqType irq_type; /* 中断类型 */
    FGpioOpsIrqHandler irq_handler;   /* 中断回调函数 */
    void *irq_args;   /* 中断回调函数参数 */
} FGpioOpsPinConfig; /* 适配 Standalone 驱动的引脚配置 */

typedef struct
{
    FGpio ctrl; /* GPIO 控制器实例 */
    FGpioPin pins[FGPIO_PORT_NUM][FGPIO_PIN_NUM]; /* GPIO 控制器下的引脚 */
    FGpioOpsPinConfig pin_config[FGPIO_PORT_NUM][FGPIO_PIN_NUM];
    boolean init_ok;
} FGpioOps;
```

2) 利用 Standalone 驱动实现 rt_pin_ops 数据结构的回调函数，PIN 设备会调用这些回调函数操作 GPIO，

```
const static struct rt_pin_ops drv_pin_ops =
{
    .pin_mode = drv_pin_mode, /* 设置 GPIO 的模式，如输入输出、中断使能等 */
    .pin_write = drv_pin_write, /* 改变 GPIO 的输出电平 */
    .pin_read = drv_pin_read, /* 获取 GPIO 的输入电平 */

    .pin_attach_irq = drv_pin_attach_irq, /* 绑定一个中断响应函数给 GPIO */
    .pin_detach_irq = drv_pin_detach_irq, /* 去绑定一个中断响应函数给 GPIO */
    .pin_irq_enable = drv_pin_irq_enable, /* 使能 GPIO 的中断 */
    .pin_get = RT_NULL
};
```

a) 上述回调函数可以使用 Standalone 驱动实现，例如使用 FGpioPinInitialize 实现 drv_pin_mode 的引脚初始化和模式设置

b) 使用 FGpioSetOutputValue 实现 drv_pin_write 的改变 GPIO 的输出电平

c) 使用 FGpioGetInputValue 实现 drv_pin_read 的获取 GPIO 的输入电平

d) 使用 FGpioOpsSetupPinIRQ 实现 drv_pin_attach_irq 的绑定中断响应函数

e) 使用 FGpioSetInterruptMask 实现 drv_pin_irq_enable 的中断使能

3) 在实现了上述功能后，将 PIN 设备的初始化流程实现，注册到 RT-Thread 的启动流程中，完成启动后，就可以通过 list kernel_device 命令看到 PIN 设备，应用程序可以通过名字获取 PIN 设备进行操作

```
int rt_hw_phytium_pin_init(void)
{
    rt_err_t ret = RT_EOK;
    ret = rt_device_pin_register("pin", &drv_pin_ops, RT_NULL); /* 注册 PIN 设备 */
    rt_kprintf("Register pin with return: %d\n", ret);
    return ret;
}
INIT_DEVICE_EXPORT( rt_hw_phytium_pin_init);
```

3.3.2 UART

1) 根据 RT-Thread 上层接口需要实现的功能，实现四个接口函数

rt_err_t uart_configure(struct rt_serial_device *serial, struct serial_configure *cfg)

```
Note：uart 配置接口，用于实现 uart 的初始化
Input:
```

```
struct rt_serial_device *device，serial 设备
struct rt_serial_configuration *configuration，serial 初始化配置
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行
```

rt_err_t uart_control(struct rt_serial_device *serial, int cmd, void *arg)

```
Note：uart 控制接口，调用该接口对 uart 进行功能上的设置
Input:
struct rt_serial_device *device，serial 设备
int cmd，设置指令，可查 RT-Thread 官方
void *arg，辅助参数，用来描述具体要配置的功能，可查 RT-Thread 官方
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行
```

int uart_putc(struct rt_serial_device *serial, char c)

```
Note：uart 按字符发送函数
Input:
struct rt_serial_device *device，serial 设备
char c，发送字符
Return:
1
```

int uart_getc(struct rt_serial_device *serial)

```
Note：uart 按字符接收函数
Input:
struct rt_serial_device *device，serial 设备
Return:
接收到的字符
```

1) 对以上接口进行注册

```
static const struct rt_uart_ops _uart_ops =
{
    uart_configure,
    uart_control,
    uart_putc,
    uart_getc,
    NULL
};
```

rt_err_t rt_hw_serial_register(struct rt_serial_device *serial, const char *name, rt_uint32_t flag, void *data)

Note：注册 qspi bus 和接口函数，用于上层调用
Input:
struct rt_serial_device *device，serial 设备
const char *name，serial 设备名称
rt_uint32_t flag，注册时开启功能
void *data，额外数据，传入定义的驱动结构体
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行

3.3.3 SPI

SPI 可以使用 RT-Thread 的 DEVICE 设备进行驱动，下面简要描述适配过程

1) 根据 RT-Thread 上层接口需要实现的功能，实现两个接口函数

rt_err_t spim_configure(struct rt_spi_device *device,struct rt_spi_configuration *configuration)

Note：spi 配置接口，用于实现 spi 的初始化
Input:
struct rt_spi_device *device，spi 设备
struct rt_spi_configuration *configuration，spi 初始化配置
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行

rt_uint32_t spim_xfer(struct rt_spi_device *device, struct rt_spi_message *message)

Note：spi 传输接口，用于实现消息的传输
Input:
struct rt_spi_device *device，spi 设备
struct rt_spi_message *message，传输的消息
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行

2) 对以上接口进行注册，并连接设备

```
static const struct rt_spi_ops spim_ops =
{
    .configure = spim_configure,
    .xfer = spim_xfer
};
```

rt_err_t rt_spi_bus_register(struct rt_spi_bus *bus, const char *name, const struct rt_spi_ops *ops)

```
Note：注册 spi bus 和接口函数，用于上层调用
Input:
struct rt_spi_bus *bus，spi bus
const char *name，spi bus 名称
const struct rt_spi_ops *ops，spi 接口注册函数
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行
```

rt_err_t rt_spi_bus_attach_device(struct rt_spi_device *device，const char *name, const char *bus_name,void *user_data)

```
Note：将 spi 设备与已注册的 spi bus 连接
Input:
struct rt_spi_device  *device，spi 设备
const char  *device_name，spi 设备名称
const char  *bus_name，spi bus 名称
void  *user_data，用户数据用于连接更多设备
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行
```

3) 实现了上述功能后，将 SPI 设备的初始化流程实现，注册到 RT-Thread 的启动流程中，完成启动后，应用程序可以通过名称获取 SPI 设备进行操作

3.3.4 QSPI

QSPI 可以使用 RT-Thread 的 DEVICE 设备进行驱动，下面简要描述适配过程

1) 根据 RT-Thread 上层接口需要实现的功能，实现两个接口函数

rt_err_t phytium_qspi_configure(struct rt_spi_device *device, struct rt_spi_configuration *configuration)

```
Note：qspi 配置接口，用于实现 qspi 的初始化
Input:
struct rt_spi_device *device，qspi 设备
struct rt_spi_configuration *configuration，qspi 初始化配置
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行
```

rt_uint32_t phytium_qspi_xfer(struct rt_spi_device *device, struct rt_spi_message *message)

```
Note：qspi 传输接口，用于实现消息的传输
```

```
Input:
struct rt_spi_device *device，qspi 设备
struct rt_spi_message *message，传输的消息
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行
```

2) 对以上接口进行注册，并连接设备

```
static struct rt_spi_ops phytium_qspi_ops =
{
    .configure = phytium_qspi_configure,
    .xfer = phytium_qspi_xfer,
};
```

rt_err_t rt_qspi_bus_register(struct rt_spi_bus *bus, const char *name, const struct rt_spi_ops *ops)

```
Note：注册 qspi bus 和接口函数，用于上层调用
Input:
struct rt_spi_bus *bus，qspi bus
const char *name，qspi bus 名称
const struct rt_spi_ops *ops，qspi 接口注册函数
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行
```

rt_err_t phytium_qspi_bus_attach_device(const char *bus_name, const char *device_name)

```
Note：将 qspi 设备与已注册的 qspi bus 连接
Input:
const char *bus_name，qspi bus 名称
const char *device_name，qspi 设备名称
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行
```

3) 实现了上述功能后，将 QSPI 设备的初始化流程实现，注册到 RT-Thread 的启动流程中，完成启动后，应用程序可以通过名称获取 QSPI 设备进行操作,使用 rt_qspi_transfer_message(struct rt_qspi_device *device, struct rt_qspi_message *message)直接进行消息收发

3.3.5 CAN

CAN 可以使用 RT-Thread 的 DEVICE 设备进行驱动，下面简要描述适配过程

4) 根据 RT-Thread 上层接口需要实现的功能，实现四个接口函数

rt_err_t _can_config(struct rt_can_device *can, struct can_configure *cfg)

Note：can 功能配置接口，调用该接口对 can 进行初始化
Input:
struct rt_can_device *can，can 设备
struct can_configure *cfg，can 初始配置
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行

rt_err_t _can_control(struct rt_can_device *can, int cmd, void *arg)

Note：can 控制接口，调用该接口对 can 进行功能上的设置，如设置模式，中断，波特率等
Input:
struct rt_can_device *can，can 设备
int cmd，设置指令，可查 RT-Thread 官方
void *arg，辅助参数，用来描述具体要配置的功能，可查 RT-Thread 官方
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行

int _can_sendmsg(struct rt_can_device *can, const void *buf, rt_uint32_t box_num)

Note：can 发送函数，调用该接口使用 can 进行发送，支持不同类型的帧
Input:
struct rt_can_device *can，can 设备
void *buf，帧缓存，内含待发送帧的信息，
rt_uint32_t box_num，邮箱数目，为 1
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行

int _can_recvmsg(struct rt_can_device *can, void *buf, rt_uint32_t fifo)

Note：can 接收函数，调用该接口使用 can 进行接收，支持不同类型的帧
Input:
struct rt_can_device *can，can 设备
void *buf，帧缓存，内含接收帧的信息，
rt_uint32_t fifo，不给予配置
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行

5) 对以上接口进行注册

```
static const struct rt_can_ops _can_ops =
{
    _can_config,
```

```
    _can_control,
    _can_sendmsg,
    _can_recvmsg,
};
```

rt_err_t rt_hw_can_register(struct rt_can_device *can, const char *name, const struct rt_can_ops *ops, void *data)

```
Note：注册 qspi bus 和接口函数，用于上层调用
Input:
struct rt_can_device *can,  can 设备
const char *name，can 设备名称
const struct rt_can_ops *ops，需注册的接口函数
void *data，额外数据
Return:
rt_err_t ，RT-Thread 错误码信息，用于判断函数是否成功执行
```

6) 实现上述功能，可使用 rt_device_find 进行设备查询，使用 rt_device_open 打开 can 设备，rt_device_control 进行 can 设备的设置，rt_device_read，rt_device_write 使用 can 进行收发。

3.3.6 SD 驱动

飞腾派上集成了 2 个 SD 控制器，一个连接 SD 卡槽支持 Micro-SD 卡，另一个连接板载的 SDIO Wifi 芯片，这里主要讨论 Micro-SD 接口及其驱动。RT-Thread 提供了 SD/MMC/SDIO 组件实现相关的协议，实现 SD 驱动的主要工作是适配这个组件，这里主要介绍一下适配这个组件的过程，

1) 创建数据结构，封装 Standalone 驱动的 SD 控制器和 DMA 描述符，作为 SD 设备的私有数据使用，

```
typedef struct
{
    FSdio *mmcsd_instance; /*  Standalone SD 控制器驱动 */
    FSdioIDmaDesc *rw_desc; /* SD 控制器 DMA 描述符 */
    rt_err_t (*transfer)(struct rt_mmcsd_host *host, struct rt_mmcsd_req *req,
FSdioCmdData * cmd_data_p);
} mmcsd_info_t;
```

2) 实现 rt_mmcsd_host_ops 中的回调函数，SD/MMC/SDIO 组件会调用这些回调函数与硬件交互

```
static const struct rt_mmcsd_host_ops ops =
{
    mmc_request_send, /* SD/MMC/SDIO 组件下发命令，包括数据读写 */
```

```
        mmc_set_iocfg, /*  SD/MMC/SDIO 组件进行控制器设置，包括时钟频率，线宽等 */
        RT_NULL, /* 获取卡的状态 */
        RT_NULL, /* 使能或者关闭 SD 控制器的中断 */
        RT_NULL, /* 发送 tuning 命令 */
    };
```

a) 上述回调函数可以使用 Standalone 驱动实现，mmc_request_send 封装了命令发送和数据的发送接收流程，使用 FSdioDMATransfer 实现，用 FSdioPollWaitDMAEnd 等待整个过程完成

b) mmc_set_iocfg 封装了初始化完成后，对控制器的配置过程，例如卡初始化完成 1 线模式切 4 线模式用 FSdioSetBusWidth 完成，400 kHz 切高速 25 MHz 或 50 MHz 用 FSdioSetClkFreq 完成

3) 然后还需要完成 SD 控制器的初始化流程，并注册到 RT-Thread 的启动流程中，SD 控制器初始化由 FSdioCfgInitialize 完成，然后注册 SD/MMC/SDIO 的 host 实例，给定 SD 卡的电压，块大小等信息，通过 mmcsd_change 提醒 RT-Thread 的 mmscsd 线程检测并开始卡的初始化

```
    int rt_phytium_mmcsd_init(void)
    {
    ...
        /* host data init */
        host->ops = &ops;
        host->freq_min = 400000;
        host->freq_max = 50000000;
        host->valid_ocr = SDIO_VALID_OCR; /* the voltage range supported is 1.65v-3.6v
*/
        host->flags = MMCSD_MUTBLKWRITE | MMCSD_BUSWIDTH_4;
        host->max_seg_size = SDIO_DMA_BLK_SZ; /* used in block_dev.c */
        host->max_dma_segs = SDIO_MAX_BLK_TRANS; /* physical segment number */
        host->max_blk_size = SDIO_DMA_BLK_SZ; /* all the 4 para limits size of one blk
tran */
        host->max_blk_count = SDIO_MAX_BLK_TRANS;
        host->private_data = private_data;

        if(RT_EOK != fsdio_ctrl_init(host))
      ...

        mmcsd_change(host);
```

4) SD 初始化成功后会生成 SD 内核设备，但是，一般来说，这个内核设备不能被直接使用，在这里，如果是 SD 内存卡，需要配合 RT-Thread 的块设备、dfs 和 posix 文件系统才能使用，对于驱动适配过程来说，只需要将 SD 内核设备挂载到一个目录即可，如下所示，将设备挂载为/目录，格式为 elm，

也就是 fat32 格式，

```
#define SD_DEIVCE_NAME "sd"

while(rt_device_find(SD_DEIVCE_NAME) == RT_NULL)
{
    rt_thread_mdelay(1);
}

if (dfs_mount(SD_DEIVCE_NAME, "/", "elm", 0, 0) == 0)
{
    LOG_I("file system initialization done!\n");
}
```

挂载完成后，在 / 目录创建文件，文件夹就是在操作 SD 卡了

3.3.7 XMAC 驱动

1) 创建数据结构，封装 Standalone 驱动的 FXmac 控制器和 rt-thread 中网络组件的 eth_device 实例，作为网卡驱动操作的主要数据对象。

```
typedef struct
{
    u8 rx_bdspace[FXMAX_RX_BDSPACE_LENGTH] __attribute__((aligned(128))); /* 接收 bd 缓冲区 */
    u8 tx_bdspace[FXMAX_RX_BDSPACE_LENGTH] __attribute__((aligned(128))); /* 发送 bd 缓冲区 */

    uintptr rx_pbufs_storage[FXMAX_RX_PBUFS_LENGTH];
    uintptr tx_pbufs_storage[FXMAX_TX_PBUFS_LENGTH];

} FXmacNetifBuffer;

typedef struct
{
    u32 instance_id;
    FXmacRtThreadInterface interface;
    u32 autonegotiation; /* 1 is autonegotiation ,0 is manually set */
    u32 phy_speed;   /* FXMAC_PHY_SPEED_XXX */
    u32 phy_duplex; /* FXMAC_PHY_XXX_DUPLEX */
} FXmacOsControl;

typedef struct
{
```

```
        struct eth_device parent; /* inherit from ethernet device */

        FXmac instance; /* Xmac controller */
        FXmacOsControl mac_config;

        FXmacNetifBuffer buffer; /* DMA buffer */
        /* queue to store overflow packets */
        PqQueue recv_q;
        PqQueue send_q;
        u32 config;          /* configuration */
        rt_uint8_t hwaddr[FXMAX_MAX_HARDWARE_ADDRESS_LENGTH]; /* MAC
address */
        struct rt_thread _link_thread; /* link detect thread */
        rt_uint8_t _link_thread_stack[LINK_THREAD_STACK_LENGTH];/* link detect
thread stack*/
    } FXmacOs;
```

2) 利用 Standalone 驱动实现 eth_device 数据结构的回调函数，rt_thread 的驱动模型会调用这些回调函数进行网卡的初始化以及接收发生网络数据包的操作。

```
    pOsMac->parent.parent.init = rt_xmac_init;
    pOsMac->parent.parent.control = rt_xmac_control;
    pOsMac->parent.eth_rx = rt_xmac_rx;
    pOsMac->parent.eth_tx = rt_xmac_tx;
```

a) 上述回调函数可以使用 Standalone 驱动实现；

b) 使用 FXmacOsInit 实现 rt_xmac_init,进行 xmac 控制器的初始化，以及重置 DMA 以及设置中断；

c) 使用 rt_xmac_control 实现 rt_device_control 获取网卡的 mac_address,将其赋值给 netif 的 hw_addr；

d) 使用 FXmacOsRX 实现 rt_xmac_rx,进行数据包接收和上传到 lwip 协议栈中；

e) 使用 FXmacOsTX 实现 rt_xmac_tx,进行数据发送以及发送缓冲区的资源释放；

3) 在实现了上述功能后，将网卡设备的初始化流程实现，注册到 RT-Thread 的启动流程中，完成启动后，就可以通过 list kernel_device 命令看到网卡设备，此时可以在 host 上通过 ping 命令与开发板进行正常数据传输。

```
    static int rt_hw_xmac_eth_init(void)
    {
        rt_err_t state = RT_EOK;
        FXmacOsControl os_config;
        FXmacOs *pOsMac;
```

```
 /* os_config initialize,need to be set manually here */
        os_config.instance_id = 0;
        os_config.interface = FXMAC_OS_INTERFACE_SGMII;
        os_config.autonegotiation = 1; /* 1 is autonegotiation ,0 is manually set */
        os_config.phy_speed =
FXMAC_PHY_SPEED_1000M;   /*FXMAC_PHY_SPEED_XXX */
        os_config.phy_duplex = FXMAC_PHY_FULL_DUPLEX; /*
FXMAC_PHY_XXX_DUPLEX */

        pOsMac = FXmacOsGetInstancePointer(&os_config);
        if (pOsMac == NULL)
        {
            LOG_E("FXmacOsGetInstancePointer is error\r\n");
            return -RT_ERROR;
        }

        state = rt_hw_xmac_init(pOsMac, os_drv_xmac0_name);
        if (RT_EOK != state)
        {
            goto __exit;
        }

__exit:
        return state;
    }
    INIT_DEVICE_EXPORT(rt_hw_xmac_eth_init);
```

3.3.8 中断支持

飞腾派上的中断控制器兼容 gicv3 规范，RT-Thread 上提供了中断控制器的驱动，并使用统一的接口提供功能支持，适配中断控制器只需要实现 RT-Thread interrupt 驱动的几个接口即可，

1) 实现 platform_get_gic_dist_base 返回 gicv3 分发器的基地址
2) 实现 platform_get_gic_redist_base 返回 gicv3 重分发器的基地址
3) 定义宏 MAX_HANDLERS，指定中断处理函数表的大小，即最多可以同时使用多少个中断号
4) 定义宏 ARM_GIC_CPU_NUM，和 CPU 核心数目一致，指定重分发器的个数
5) 定义宏 GIC_IRQ_START，指定最小的中断号

3.4 RT-Thread 内核调试方法

内核适配到稳定工作的过程中可能会遇到各种问题，下面总结了一部分常用的内核调试方法，可以帮助加快问题排查的速度，

1) FinSH 自带命令‘ps’

在成功启动 rt-thread 后，可以输入‘ps’命令后，查看当前系统中正在运行的进程（线程）的信息。该命令可以显示每个进程的 ID、优先级、堆栈使用情况等详细信息。

使用 ps 命令可以帮助你了解当前系统中的进程状态，以及它们所占用的系统资源。这对于调试和性能优化非常有帮助。

```
msh />ps
thread           cpu bind pri  status      sp     stack size max used left tick  error
---------------- --- ---- ---  ------- ---------- ---------- ------  ---------- ---
core1_test       N/A   1   20  suspend 0x00000318 0x00001000    24%   0x0000001e EINTRPT
core0_test       N/A   0   20  suspend 0x00000318 0x00001000    27%   0x0000001e EINTRPT
tshell             0   2   20  running 0x000004b8 0x00001000    62%   0x00000009 OK
sys workq        N/A   2   23  suspend 0x000002a8 0x00002000    08%   0x0000000a OK
tsystem          N/A   2   30  suspend 0x00000308 0x0000a000    01%   0x00000020 EINTRPT
tidle1             1   1   31  running 0x00000228 0x0000a000    02%   0x00000012 OK
tidle0           N/A   0   31  ready   0x00000258 0x0000a000    06%   0x00000008 OK
timer            N/A   2    4  suspend 0x00000298 0x00001000    21%   0x00000009 OK
```

图 3-1 ps cmd

2) 调用 rt_kprintf 打印文件名与所在行数

通过 rt_kprintf 在函数执行过程中打印当前文件名与所在行数来确认函数执行位置与出现 bug 的位置。

```
rt_kprintf("File name: %s ; Line: %d \n", __FILE__, __LINE__);
```

```
File name: applications\main.c ; Line: 43
```

图 3-2 rt_kprintf debug

3) 使用 backtrace

Phytium bsp 支持使用 backtrace 进行调试，backtrace 是一个用于追踪函数调用堆栈的过程。它记录了程序在执行过程中每个函数的调用关系，从而可以提供关于程序在特定时刻执行流程的详细信息。

通过 backtrace，可以获取调用堆栈中的函数名、调用顺序和地址等信息。这在调试和故障排查过程中非常有用，特别是当程序发生错误或崩溃时。

使用 backtrace 可以帮助开发人员追踪代码中的执行路径，以了解错误发生的位置和调用上下文。这有助于快速定位问题，进行错误修复和代码优化。另外，调试器和异常处理机制也经常使用 backtrace 来显示问题发生的位置和上下文信息，帮助开发人员理解和解决问题，使用方法如下。

a) 在程序发生错误或奔溃时，会出现以下打印

```
please use: addr2line -e rtthread.elf -a -f 0x00000000800852a4 0x00000000800866d8 0x000000008008085c 0x00000000800c4b18 0x0000000080080dd8 0x0000000080080e34 0x00000000800aa250
exception info:
esr.EC :0x25
esr.IL :0x01
esr.ISS:0x00000006
epc    :0x00000000800a9cfc
Data abort
fault addr = 0x0000000000000000
abort caused by read instruction
Translation fault, second level
Execption:
X00:0x0000000000000000 X01:0x0000000000000013 X02:0x0000000080081b18 X03:0x0000000000000013
X04:0x00000000800c37b8 X05:0x0000000000000012 X06:0x00000000ffffffff X07:0x0000000000000021
X08:0x0000000000000008 X09:0x0000000000000009 X10:0x000000000000000a X11:0x000000000000000b
X12:0x000000000000000c X13:0x000000000000000d X14:0x000000000000000e X15:0x000000000000000f
X16:0x0000000000000010 X17:0x0000000000000011 X18:0x0000000000000012 X19:0x0000000080080de4
X20:0x00000000800bb898 X21:0x0000000000000015 X22:0x0000000000000016 X23:0x0000000000000017
X24:0x0000000000000018 X25:0x0000000000000019 X26:0x000000000000001a X27:0x000000000000001b
X28:0x0000000000000000 X29:0x00000000800edf38 X30:0x00000000800a9d6c
SP_EL0:0x0000000000000000
SPSR  :0x00000000600003c5
EPC   :0x00000000800a9cfc
current:
        thread           pri  status      sp     stack size max used left tick  error
---------------- ---  ------- --------- ---------- ------  ---------- ---
```

图 3-3 backtrace 打印

b) 按照提示将该行内容复制，在终端进入到 elf 文件所在文件夹，粘贴后执行命令即可，下图所示为程序崩溃前调用的函数名称与相关信息

```
FtOut32
D:\phytium-dev-linux-x64\rt-thread\bsp\phytium\aarch64/D:\phytium-dev-linux-x64\rt-thread\bsp\phytium\libraries\standalone\common/fio.h:71
0x00000000800866d8
FQspiFlashWriteData
D:\phytium-dev-linux-x64\rt-thread\bsp\phytium\aarch64/D:\phytium-dev-linux-x64\rt-thread\bsp\phytium\libraries\standalone\drivers\qspi\fqspi/fqspi_flash.c:637 (discriminator 3)
0x000000008008085c
phytium_qspi_xfer
D:\phytium-dev-linux-x64\rt-thread\bsp\phytium\aarch64/D:\phytium-dev-linux-x64\rt-thread\bsp\phytium\libraries\drivers/drv_qspi.c:144
0x00000000800c4b18
rt_qspi_transfer_message
D:\phytium-dev-linux-x64\rt-thread\bsp\phytium\aarch64/D:\phytium-dev-linux-x64\rt-thread\components\drivers\spi/qspi_core.c:87
0x0000000080080dd8
WriteCmd
D:\phytium-dev-linux-x64\rt-thread\bsp\phytium\aarch64/D:\phytium-dev-linux-x64\rt-thread\bsp\phytium\libraries\drivers/drv_qspi.c:314
0x0000000080080e34
qspi_thread
D:\phytium-dev-linux-x64\rt-thread\bsp\phytium\aarch64/D:\phytium-dev-linux-x64\rt-thread\bsp\phytium\libraries\drivers/drv_qspi.c:326
0x00000000800aa250
_thread_start
D:\phytium-dev-linux-x64\rt-thread\bsp\phytium\aarch64/D:\phytium-dev-linux-x64\rt-thread\libcpu\aarch64\common/context_gcc.S:83
```

图 3-4 backtrace 使用方法

4 RT-Thread 综合应用

前面的章节介绍了 RT-Thread 的基本功能，以及如何将 RT-Thread 适配到飞腾派上，本章主要介绍 RT-Thread 三个方面的综合应用，分别是文件系统、网络协议和 RT-Smart。文件系统方面，这里介绍了如何使用 RT-Thread 内核中提供的 ulog 组件，将系统运行日志保存在文件系统中；网络协议栈方面，这里介绍了如何使用 RT-Thread 内核中提供的 lwip 组件，在局域网中搭建一个 tftp 服务，支持局域网其他设备上传和下载文件；RT-Smart 方面，介绍了如何使用 RT-Smart 提供的 busybox 用户态组件，形成一个类似 linux 的使用环境。

4.1 文件系统与 ulog 日志使用

4.1.1 ulog 日志系统

ulog 是一个非常简洁、易用的 C/C++ 日志组件，支持在中断中输出调试信息。使用 ulog 需要打开 RT_USING_ULOG 配置：

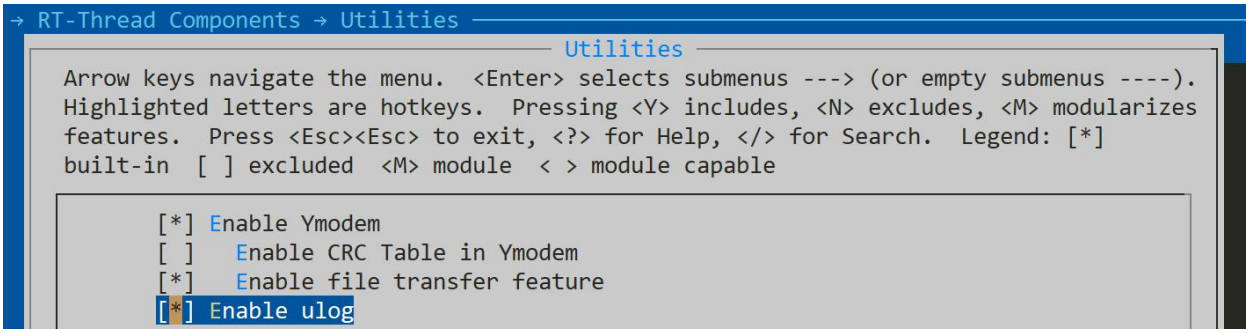


图 4-1 ulog enable

ulog 所使用的前端 API 与 RT-Thread 标准调试信息打印接口 LOG_X 一致，使能 ulog 后，所有 LOG_X 接口将被替换为 ulog 接口，并输出 ulog 调试信息。

通过修改 LOG_TAG 宏与 LOG_LVL 宏，可以控制本源文件出现的 LOG_X 输出的标签与日志级别，也可以在 msh 控制台输入 ulog_lvl、ulog_tag 等命令来动态控制输出的日志级别与过滤标签，详见 ulog 日志（rt-thread.org）。

4.1.2 文件系统的 POSIX 接口

4.1.3 配置 ulog 将日志保存成文件

ulog 支持前后端分离，在前端统一 LOG_X 的接口下，可以将日志数据输出到各种不同的后端，如：控制台、文件系统等。将文件系统作为后端注册到 ulog 中，即可将 ulog 日志保存到文件系统目录下。自 RT-Thread V4.1.0 以后，有关 ulog 输出到文件系统的软件包已移动到 RT-Thread 内部。在注册前，需要对 ulog 进行如下配置：

```
[*] Enable ulog
      The static output log level. (Debug)  --->
[*]   Enable ISR log.
[*]   Enable assert check.
(256) The log's max width.
[*]   Enable async output mode.
(2048)  The async output buffer size.
[*]     Enable async output by thread.
(4096)    The async output thread stack size.
(30)      The async output thread stack priority.
      log format  --->
[*]   Enable console backend.
[*]   Enable file backend.
[*]   Enable runtime log filter.
[ ]   Enable syslog format log and API.
```

图 4-2 ulog config

各配置说明如下：

```
[*] Enable ulog                  /* 使能 ulog */
      The static output log level./* 选择静态的日志输出级别。选择完成后，比设定级别低的日志（这里特指使用 LOG_X API 的日志）将不会被编译到 ROM 中 */
[ ]   Enable ISR log.            /* 使能中断 ISR 日志，即在 ISR 中也可以使用日志输出 API */
[*]   Enable assert check.       /* 使能断言检查。关闭后，断言的日志将不会被编译到 ROM 中 */
(128) The log's max width.       /* 日志的最大长度。由于 ulog 的日志 API 按行作为单位，所以这个长度也代表一行日志的最大长度 */
[ ]   Enable async output mode.  /* 使能异步日志输出模式。开启这个模式后，日志不会立刻输出到后端，而是先缓存起来，然后交给日志输出线程（例如：idle 线程
      log format  --->           /* 配置日志的格式，例如：时间信息，颜色信息，线程信息，是否支持浮点等等 */
[*]   Enable console backend.    /* 使能控制台作为后端。使能后日志可以输出到控制台串口上。建议保持开启。 */
[ ]   Enable file backend.       /* ulog 的文件后端 */
[ ]   Enable runtime log filter. /* 使能运行时的日志过滤器，即动态过滤。使能后，日志将支持按标签、关键词等方式，在系统运行时进行动态过滤。 */
[ ]   Enable syslog format log and API.  /* 启用 syslog 格式日志和 API */
```

图 4-3 ulog 配置说明

ulog 会将所有 LOG_X 输出的文本信息保存到预设的.log 文件下。

```
sys
文件 编辑 查看
01-01 08:00:00 W/time: Cannot find a RTC device!
01-01 08:00:00 D/NO_TAG: In FGpioOpsSetupCtrlIRQ() -> cpu_id 1, irq_num 189

01-01 08:00:00 I/NO_TAG: GPIO-4 report irq by controller
01-01 08:00:00 I/GPIO: ==> Set GPIO-4-a-12 to low
01-01 08:00:00 I/GPIO: <== Get GPIO-4-a-11 as low
01-01 08:00:00 W/time: Cannot find a RTC device!
01-01 08:00:00 D/NO_TAG: In FGpioOpsSetupCtrlIRQ() -> cpu_id 1, irq_num 189

01-01 08:00:00 I/NO_TAG: GPIO-4 report irq by controller
01-01 08:00:00 I/GPIO: GPIO-4-a-11, Interrupt Asserted!!!
01-01 08:00:00 I/GPIO: Intr!
01-01 08:00:00 I/GPIO: ==> Set GPIO-4-a-12 to high
01-01 08:00:00 I/GPIO: <== Get GPIO-4-a-11 as high
01-01 08:00:00 D/NO_TAG: In FGpioOpsSetupCtrlIRQ() -> cpu_id 1, irq_num 189

01-01 08:00:00 I/NO_TAG: GPIO-4 report irq by controller
01-01 08:00:00 I/GPIO: ==> Set GPIO-4-a-12 to low
01-01 08:00:00 I/GPIO: <== Get GPIO-4-a-11 as low
01-01 08:00:00 W/time: Cannot find a RTC device!
01-01 08:00:00 D/NO_TAG: In FGpioOpsSetupCtrlIRQ() -> cpu_id 1, irq_num 189

01-01 08:00:00 I/NO_TAG: GPIO-4 report irq by controller
01-01 08:00:00 I/GPIO: GPIO-4-a-11, Interrupt Asserted!!!
01-01 08:00:00 I/GPIO: Intr!
01-01 08:00:00 I/GPIO: ==> Set GPIO-4-a-12 to high
01-01 08:00:00 I/GPIO: <== Get GPIO-4-a-11 as high
```

图 4-4 ulog 文件输出

4.2 网络协议与 tftp 服务

4.2.1 tftp 协议

TFTP 是一个传输文件的简单协议，它基于 UDP 协议而实现，我们可以使用 tftp 进行主机和开发板之间的文件传输。使用 tftp 需要先将源文件加入到编译包含路径中：

```
components > net > lwip > lwip-2.1.2 > SConscript > ...
233   src += lwipsixlowpan_SRCS
234
235   src += lwipcore4_SRCS
236   src += lwiptftp_SRCS
237   path = [cwd + '/src/include',
238       cwd + '/src/include/netif']
```

图 4-5 tftp

4.2.2 文件传输演示

将编译后的镜像烧录进开发板，在开发板上输入 tftp_server 打开 tftp 服务，此时可以通过 tftp 协议进行主机和开发板之间的网络文件传输：

```
 \ | /
- RT -     Thread Operating System
 / | \     5.0.1 build Aug  2 2023 16:37:33
 2006 - 2022 Copyright by RT-Thread team
lwIP-2.1.2 initialized!
netif: netmask of interface e0 set to 255.255.255.0
netif: GW address of interface e0 set to 192.168.4.1
netif_set_ipaddr: netif address being changed
FXMAC OS Init Success!
netif: setting default interface e0
netif: added interface e0 IP addr 192.168.4.10 netmask 255.255.255.0 gw 192.168.4.1
Xmac0 Initiailized!
[I/sal.skt] Socket Abstraction Layer initialize success.
rt_hw_secondary_cpu_up is processing
N: Standard Service Call: 0xc4000003
N: PSCI: pwr domain on 0x201
Parametr : core map  --- 0x311
N: PSOSTAT_REG = 8a
N: scmi_power_domain_set.
N: pwr_domain_on_finish
N: the core security level: 0x0 normal
V: HANDLER: callback had not init
N: pwr_domain_olink iph
                        Hi,e.ore0
ond core jump to os.
arm_gic_redist_init is over rt_hw_cpu_id() is 1
call cpu 1 on success
Hi, core1

msh />tft
tftp_server
msh />tftp_server
TFTP server start successfully.
msh />
```

图 4-6 rtthread 启动

以下图片为我们演示了 tftp put 指令，该指令可以将主机上的文件通过网络传输到开发板中：

```
phytium@DESKTOP-R6BG2D0:/mnt/c/Users/123/Desktop/test/230725_rtthrread/bsp/phytium/aarch64$ tftp -m binary 192.168.4.10
tftp> put Kconfig
tftp>
```

```
msh />ls
Directory /:
msh />ls
Directory /:
Kconfig             1243
msh />
```

图 4-7tftp put

以下图片为我们演示了 tftp get 指令，该指令可以将开发板上的文件通过网络传输到主机上：

```
msh />echo hello_phytium test
msh />ls
Directory /:
test                13
Kconfig             1243
```

```
tftp> get test
tftp> q
phytium@DESKTOP-R6BG2D0:/mnt/c/Users/123/Desktop/test/230725_rtthrread/bsp/phytium/aarch64$ cat test
hello_phytiumphytium@DESKTOP-R6BG2D0:/mnt/c/Users/123/Desktop/test/230725_rtthrread/bsp/phytium/aarch
```

图 4-8 tftp get

4.3 RT-Smart 用户态与 Busybox 工具

BusyBox 是一个开源项目，它提供了大约 400 个常见 UNIX/Linux 命令的精简实现，包括 ls、mv、ln、mkdir、more、ps、gzip、bzip2、tar 和 grep，甚至还集成了一个 http 服务器和 telnet 服务器，对于小尺寸操作系统来说，Busybox 是一个极佳的最小环境。RT-Smart 是基于 RT-Thread 操作系统上的混合操作系统，提供了用户态支持，Busybox 是 RT-Smart 提供的一个用户态程序。

4.3.1 构建 RT-Smart 用户态程序

构建 RT-Smart 用户态程序可以使用仓库

[RT-Thread/userapps: https://github.com/RT-Thread/userapps.git]

构建用户态程序前要配置 xmake 工具，可以按照上面的仓库提供的方法配置，也可以按照 xmake 官方的方法源码编译

```
bash <(wget https://xmake.io/shget.text -O -)
```

4.3.2 Busybox 工具的使用

构建完成后，将 rootfs 烧写到 SD 卡中，启动开发板成功后，就可以使用 Busybox 工具。

5 附录

A 更新记录

发布日期	说明
2023-08-04	初稿